EasySave Technical Support Guide

Version 1.0 — For ProSoft Support Staff Only

Overview

This document is intended for ProSoft support staff. It covers everything needed to diagnose and resolve issues with EasySave v1.0: minimum system requirements, file locations, architecture and design patterns, toolchain details, log and state file formats, and the most common problems customers report.

EasySave is a .NET 8 console application that lets end-users create up to five backup jobs (full or differential), run them individually or in batch, and track activity through real-time JSON log files. The EasyLog component is shipped as a separate DLL so its logging logic can be reused by other ProSoft products without modification to the main executable.

Minimum System Requirements

Item	Detail
Operating System	Windows 10 or later (64-bit)
Runtime	.NET 8.0 — must be installed separately if not already present on the machine
Disk Space	At least 50 MB free for the application; additional space for log and state files
Permissions	Read access to source folders; Write access to target folders and %APPDATA%\EasySave\
Network	Required only when source or target paths point to a UNC network share
IDE (Dev)	Visual Studio 2022 or later (required for building from source)
SDK (Dev)	.NET 8.0 SDK (required for compilation and unit tests)

File Locations

All runtime files are stored under the Windows user profile (%APPDATA%) so the application never needs administrator privileges and functions correctly on shared or multi-user servers.

File	Default Path	Notes
Executable	EasySave.exe — wherever the customer installed it	No fixed path; customer-defined
Job config	%APPDATA%\EasySave\jobs.json	Created automatically on first launch
Real-time state	%APPDATA%\EasySave\state.json	Overwritten live during every backup
Daily log files	%APPDATA%\EasySave\Logs\YYYY-MM-DD.json	New file created each calendar day
Language files	.\Resources\string-en.json and string-fr.json	Must sit next to the executable
EasyLog DLL	EasyLog.dll — same folder as EasySave.exe	Loaded at runtime via standard .NET assembly resolution

Note: *%APPDATA% resolves to C:\Users\<username>\AppData\Roaming on a standard Windows installation.*

Architecture & Design Patterns

Understanding the architecture is essential for diagnosing non-obvious bugs. EasySave v1.0 uses several well-known software design patterns, each chosen to keep the codebase easy to extend for future versions.

MVVM (Model–View–ViewModel)

The entire application is structured around MVVM, even though v1.0 is a console app. This separation was mandatory per the specification because a graphical v2.0 is planned.

Layer	Class / File	Responsibility
Model	BackupJob, BackupManager, ConfigManager, StateManager, LocalizationManager	All business logic, persistence, and domain state
ViewModel	MainViewModel	Exposes commands and properties to the View; contains CLI parsing logic
View	ConsoleView	Renders menus, reads user input, calls ViewModel commands only — no business logic

If a bug manifests in the UI (wrong text, wrong menu behaviour), look in ConsoleView. If a backup is producing incorrect results, look in the Model layer. MainViewModel should contain no I/O or file-system calls.

Strategy Pattern (Backup Algorithms)

The two backup types are implemented as interchangeable strategy objects behind a common interface:

```
interface IBackupStrategy { void ExecuteBackup(string source, string target, BackupJob
job); }
class FullBackupStrategy    : IBackupStrategy  // copies every file unconditionally
class DifferentialBackupStrategy : IBackupStrategy  // copies only new/modified files
```

BackupJob holds a reference to whichever strategy was chosen at creation time. BackupManager instantiates the correct strategy when loading jobs from jobs.json (using the IsDifferential flag). Adding a new backup type in a future version requires only a new class implementing IBackupStrategy — no changes to BackupJob or BackupManager.

Observer Pattern (Real-Time State Updates)

BackupJob exposes a standard .NET event:

```
public event EventHandler ProgressUpdated;
```

BackupManager subscribes StateManager.OnJobProgressUpdated to this event before each job runs, then unsubscribes in a finally block to prevent memory leaks. This means state.json is written automatically after every file copy without BackupJob needing to know anything about StateManager.

If state.json is not updating during a backup, the first thing to check is whether the event subscription is executing — a breakpoint or log line in OnJobProgressUpdated will confirm this.

Singleton Pattern (EasyLogger & LocalizationManager)

Two components are implemented as thread-safe singletons:

Singleton	Class	Access point
Logger	EasyLogger (in EasyLog.dll)	EasyLogger.Instance.WriteLog(entry)
Translations	LocalizationManager	LocalizationManager.Instance.GetString(key)

EasyLogger uses Lazy<T> for thread-safe lazy initialisation. It also holds an internal object lock (_writeLock) so concurrent file writes from multiple jobs do not corrupt the JSON output. LocalizationManager uses a static readonly field for the same thread-safety guarantee.

Because EasyLogger is a singleton, its log directory is set once at startup. If you need to redirect logs to a different path in a test or diagnostic scenario, use the overloaded constructor new EasyLogger(customPath) — this bypasses the singleton and creates a standalone instance suitable for unit testing.

Repository / Interface Segregation (IConfigManager)

Job persistence is abstracted behind IConfigManager:

```
interface IConfigManager { void SaveJobs(List<BackupJob>); List<JobSaveData> LoadJobs();
}
```

The concrete implementation ConfigManager writes to jobs.json in %APPDATA%. BackupManager accepts an optional IConfigManager in its constructor — if none is provided, it falls back to ConfigManager. This design allows unit tests (see EasySave.Tests) to inject a mock config manager without touching the filesystem.

Toolchain & Development Environment

Tool	Version / Detail	Purpose
Visual Studio	2022 or later	Primary IDE for all C# development; required by ProSoft policy
C# / .NET	.NET 8.0 — C# 12	Language and runtime for both EasySave.exe and EasyLog.dll
GitHub	github.com — Git version control	Source control, branch management, pull requests, change tracking
ArgoUML	Recommended UML editor	Class diagrams and sequence diagrams (submitted 24 h before each deliverable)
MSTest / xUnit	EasySave.Tests project	Unit tests covering BackupManager, strategy logic, and EasyLogger
System.Text.Json	.NET built-in	All JSON serialisation/deserialisation for jobs.json, state.json, log files, and resource strings
System.Diagnostics.Stopwatch	.NET built-in	Measures per-file transfer time in milliseconds (stored in transferTimeMs)

Project Structure

The solution (EasySave.slnx) contains three projects:

Project	Output	Description
EasySave	EasySave.exe	Main console application — Models, ViewModel, View, Resources
EasyLog	EasyLog.dll	Reusable logging library — EasyLogger singleton and LogEntry model
EasySave.Tests	Test assembly	Unit tests for BackupManager, strategy behaviour, and logger correctness

Configuration File Format (jobs.json)

Created automatically on first launch and updated whenever a job is added or removed.

Field	Type	Description
Name	string	The label given to the job by the user.
Source	string	Full path to the source folder — local or UNC format.
Target	string	Full path to the destination folder.
IsDifferential	boolean	true = differential backup (copy new/modified files only); false = full backup (copy all files).

Daily Log File Format (YYYY-MM-DD.json)

A new file is created in %APPDATA%\EasySave\Logs\ each calendar day. Every file copied during a backup adds one JSON entry. Files are written in real time, so the log can be inspected while a backup is in progress. The EasyLogger singleton uses an internal lock to prevent write corruption when multiple jobs run.

Field	Type	Description
timestamp	ISO 8601 string (UTC)	Date and time the file was copied; written in UTC.
backupName	string	Name of the job that triggered the copy.
sourceFilePath	string	Full UNC path to the original file.
targetFilePath	string	Full UNC path to the destination copy.
fileSize	number (bytes)	File size in bytes, read via FileInfo before copying.
transferTimeMs	number (ms)	Copy duration in milliseconds via Stopwatch. Negative value means the copy threw an exception.

Real-Time State File Format (state.json)

Updated live during every backup. Contains one entry per configured job. StateManager persists its in-memory dictionary of all job states as a JSON array, so historical job entries are not lost between runs.

Field	Type / Format	What it tells you
Name	string	The job name.
LastUpdate	yyyy-MM-dd HH:mm:ss (local)	When this entry was last written.
Status	string enum	Inactive | Active | Completed | Error — set by the Observer callback.
TotalFiles	integer	Total files counted in the source directory at the start of the run.
TotalSize	long (bytes)	Total bytes of all files to be processed.
Progress	integer (0–100)	Percentage of files already copied, calculated as (TotalFiles - FilesRemaining) / TotalFiles * 100.
FilesRemaining	integer	Decremented after each file, including skipped files in a differential run.
SizeRemaining	long (bytes)	Bytes not yet processed; decremented per file.
CurrentSource	string	Full path to the file currently being read.
CurrentDestination	string	Full path to where that file is being written.

Unit Tests

The EasySave.Tests project contains automated tests covering the core logic. Running the tests is the fastest way to confirm that a local build is healthy before deploying to a customer machine.

Test class	What is covered
BackUpManagerTests	Job creation, maximum-job limit (5), loading from IConfigManager mock, ExecuteJob and ExecuteAll orchestration.
StrategyTests	FullBackupStrategy copies all files; DifferentialBackupStrategy skips unchanged files and copies new/modified ones. Uses temporary directories.
LogTests	EasyLogger writes valid JSON to a temp directory; negative transferTimeMs is preserved correctly; concurrent writes do not corrupt the file.

To run the tests from the command line:

```
cd EasySave && dotnet test EasySave.Tests/EasySave.Tests.csproj
```

Common Issues & Resolutions

Symptom	Diagnosis & Resolution
App starts then closes immediately	The .NET 8.0 runtime is not installed. Ask the customer to download it from microsoft.com/dotnet. Verify the installed version with: dotnet --version
Jobs disappear after restart	jobs.json is missing or contains malformed JSON. Check %APPDATA%\EasySave\. If the file is corrupt, delete it — the app recreates a clean empty array on next launch. ConfigManager.LoadJobs() catches parse exceptions and silently returns an empty list.
state.json is not updating during backup	Either (a) the process lacks write access to %APPDATA%\EasySave\ — check NTFS permissions — or (b) the Observer event was never subscribed. If building from source, confirm BackupManager.ExecuteJob() subscribes job.ProgressUpdated before calling job.Execute().
Log files are not appearing	Same as above: check write access to %APPDATA%\EasySave\Logs\. Also confirm the Logs subfolder exists (created by EasyLogger constructor on first run). The EasyLogger._writeLock ensures concurrent writes are safe, so race conditions are not the cause.
A backup runs but no files are copied	For full backup: check that the source path exists and is not empty. For differential backup: all files may already be up to date (LastWriteTime on source <= target). UNC network paths may also be unreachable — confirm connectivity.
transferTimeMs shows negative values	A negative value means File.Copy() threw an exception. Inspect sourceFilePath in that log entry and check file permissions, disk space on the target drive, or whether the file is locked by another process.
Interface is in the wrong language	Language preference is not persisted between sessions (by design in v1.0). The customer must select it each time via menu option 3. If the language does not change at all, confirm that string-en.json and string-fr.json exist in the Resources\ folder next to the executable. LocalizationManager silently falls back to the key name if the file is missing.
EasyLog.dll not found at startup	EasyLog.dll must be in the same directory as EasySave.exe. It is a .NET class library referenced at compile time; .NET resolves it via standard probing paths. Verify the DLL is present with: dir /b EasySave.exe EasyLog.dll
IsDifferential flag is wrong in jobs.json	ConfigManager.SaveJobs() determines the flag at save time using a type check: job.GetStrategy() is DifferentialBackupStrategy. If jobs were added via a code path that does not persist to disk immediately, the flag may not match what is displayed in the menu.

Localisation

All user-facing strings are externalised into JSON resource files loaded by the LocalizationManager singleton. The current supported languages are English (EN) and French (FR).

File	Path	Encoding
string-en.json	.\Resources\string-en.json	UTF-8
string-fr.json	.\Resources\string-fr.json	UTF-8

If a translation key is missing from the file, LocalizationManager.GetString() returns the key itself wrapped in square brackets (e.g., [MsgJobFinished]). This makes missing translations immediately visible in the UI without crashing the application.

Known Limitations (v1.0)

- Maximum of 5 backup jobs per installation (hard-coded limit in BackupManager).
- Jobs run sequentially. There is no parallel execution in v1.0.
- Language preference resets on each launch; no persistence for the language setting.
- Differential backup uses LastWriteTime only; file content checksums are not compared.
- No built-in encryption or compression of backup files.
- The EasyLog DLL writes plain JSON; it does not support remote or structured logging sinks.

- CLI mode (EasySave.exe 1-3) executes jobs synchronously and exits without showing the interactive menu.